

MediCare

PROJECT PLAN & TECHNICAL ARCHITECTURE

Department of Computer Science & Engineering, SRM University

1. Project Overview

MediCare is a console-based, menu-driven application designed to digitize hospital appointment booking and prescription management. The system allows doctors to publish their available time slots and manage appointments, while enabling patients to browse doctors by specialization, book and cancel appointments, and view their prescriptions. The application is built with a strict modular architecture separating user interface, business logic, and database operations.

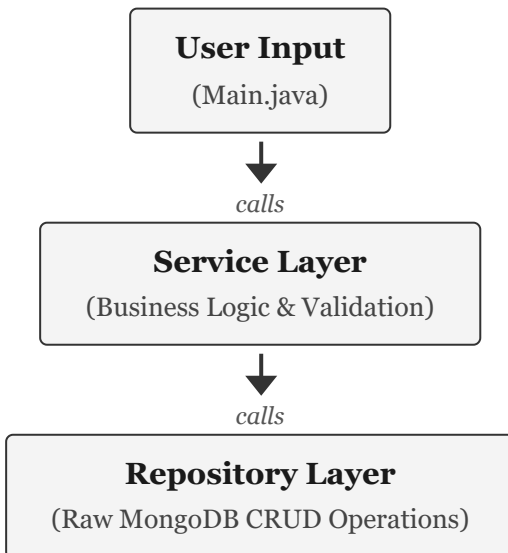
2. Technology Stack & Setup

- **Programming Language:** Java 17 (JDK 17)
- **Database:** MongoDB Atlas (Cloud Database)
- **Build Automation:** Apache Maven (3.9.x)
- **Version Control:** Git & GitHub
- **IDE:** Visual Studio Code (with Microsoft Java Extension Pack)

Note: Database connection strings (MONGO_URI) are managed securely via local system environment variables and are never hardcoded or committed to the repository.

3. System Architecture

The application follows a strictly layered architectural pattern. Each layer is only aware of the layer directly below it. The UI never interacts directly with the database, ensuring clean separation of concerns and maintainability.





MongoConnection Utility
(MongoDB Link)

4. Data Models & File Structure

The codebase is organized into distinct packages mapping to the system's logical layers:

model/ (Data Blueprints)

These classes define the structure of the data and contain only fields, getters, and setters with no business logic.

- **User.java:** Abstract base class containing ID, name, email, and password.
- **Doctor.java:** Extends User. Contains a list of appointment IDs managed by the doctor.
- **Patient.java:** Extends User. Contains a list of appointment IDs placed by the patient.
- **Appointment.java:** Represents a booked appointment (ID, doctorId, patientId, date, timeSlot, status).
- **Prescription.java:** Represents a prescription issued by a doctor (ID, appointmentId, doctorId, patientId, diagnosis, medicines, timestamp).

repository/ (Database Access)

Responsible for converting Java objects to MongoDB Documents and handling raw DB queries.

- **DoctorRepository.java & PatientRepository.java:** Handle user CRUD operations and lookups by specialization.
- **AppointmentRepository.java:** Handles booking, slot-availability checks, status updates, and appointment history queries.
- **PrescriptionRepository.java:** Manages prescription records and retrieval.

service/ (Business Logic)

Handles decisions, calculations, and multi-step transactions.

- **DoctorService.java:** Validates data (e.g., rejecting empty specialization fields) before calling DoctorRepository.
- **PatientService.java:** Handles patient registration and profile validation.
- **AppointmentService.java:** Core transactional logic. Verifies that the requested slot is free, inserts the appointment, and links it to the doctor and patient. Enforces rules such as rejecting past dates and blocking cancellation of completed appointments.
- **PrescriptionService.java:** Attaches prescriptions only to completed appointments.

util/ & exception/

- **util/MongoConnection.java:** Singleton class managing the connection lifecycle to MongoDB Atlas.

- **exception/SlotUnavailableException.java & PatientNotFoundException.java:**
Custom runtime errors for application-specific failure states.

5. Implementation Plan

The modules are implemented sequentially in dependency order. Each step builds on the previous one, and each layer is tested before moving on.

1. Project setup (Maven pom.xml, .gitignore); util/MongoConnection.java; all Data Models (User, Doctor, Patient, Appointment, Prescription).
2. DoctorRepository.java; PatientRepository.java; DoctorService.java.
3. AppointmentRepository.java; AppointmentService.java; SlotUnavailableException.java.
4. PrescriptionRepository.java; PrescriptionService.java; PatientNotFoundException.java.
5. Final wiring in Main.java (menu logic & try/catch); end-to-end testing of the full flow; release.

6. Development & Deployment Workflow

To prevent code collisions and ensure smooth collaboration, the team utilizes a strict Git branching strategy. No developer commits directly to the main branch. Individual features are developed on dedicated feature branches (e.g., `feature/appointment-booking`), reviewed via pull requests, and merged into main only after the integration test passes.

- Each feature branch is short-lived and is pulled from an up-to-date main before opening a pull request.
- Code reviews focus on layering discipline: the service layer must never import from the UI, and repositories must never be called directly from Main.java.
- The shared GitHub repository is the single source of truth; team members push their branches daily to avoid stale forks.
- Releases are tagged (e.g., `v1.0`) once the full menu flow — register, login, book, cancel, complete, prescribe — passes end-to-end.
- Final demo: set the `MONGO_URI` environment variable locally, build with `mvn clean package`, and run the application (`mvn exec:java` or launch Main.java from VS Code). The connection to MongoDB Atlas is established on startup.

Subject: Advanced Programming Practice

Submitted by

Prabhat Arora

Reg. No.: 06

AI/ML A